

Kubernetes Cluster Installation on On-Premise Servers

Author: Jash Hamirani

Date: 22/08/2025

Environment: Ubuntu 24.04

Kubernetes Version: v1.33

CRI: containerd

CNI: Calico / Cilium

Kubernetes plays an integral role in modern-day software deployment. It's the keystone in DevOps, offering a stable and robust platform for automating the deployment and management of containerized applications.

Also popularly known as K8s, Kubernetes offers numerous benefits for your microservices infrastructure. These include high availability (automatic restart or self-healing of failed containers), auto-scaling containers, and efficient resource management.

This tutorial will demonstrate how to install Kubernetes on [Ubuntu 24.04](#) using a master node and two worker nodes. After that, we will initialize a cluster and test its functionality by deploying a simple Nginx application across all nodes.

Prerequisites

Here are the salient requirements for the installation of Kubernetes:

- Minimum of 2 instances of Ubuntu 24.04. One will act as the master node and the remaining as the worker node. For this guide, we will work with two worker nodes.
- SSH access to all the instances with sudo users configured.
- Minimum of 2GB RAM and 2 vCPUs.
- Minimum 20GB of free hard drive space.

1. Update and upgrade system packages:

Begin by ensuring that your system is up to date. Open a terminal and execute the following commands:

```
# sudo apt update && sudo apt upgrade -y
```

2. Disable swap (Kubernetes requires swap to be disabled):

To enhance Kubernetes performance, disable swap and set essential kernel parameters.

```
# sudo swapoff -a  
sudo sed -i 's/^swap / s/^/#/' /etc/fstab
```

3. Configure hostnames (all nodes)

Start by updating the `/etc/hosts` files and **specifying the Private IP addresses and hostnames** for each node in the cluster. It would be for **hostname resolution**

On master node

```
sudo hostnamectl set-hostname master
```

On worker node

```
sudo hostnamectl set-hostname server1
```

```
127.0.0.1 localhost  
[redacted] master  
[redacted] server1
```

4. Verify hostname

To verify the hostname in each node, run the command:

```
# hostname
```

or

```
# ping -c 3 master // from worker & vice-versa
```



```
server1:~$ hostname  
server1
```

5. Add Kernel Parameters to load Containerd Modules (on all servers)

Containerd is the standard container runtime for Kubernetes. For the seamless running of this runtime, it is recommended that the **overlay** and **br_netfilter** kernel modules be loaded.

To enable and load these modules, run the following commands:

```
# sudo modprobe overlay
```

```
# sudo modprobe br_netfilter
```

Create a configuration file as shown and specify the modules to load them permanently.

```
# sudo tee /etc/modules-load.d/k8s.conf <<EOF  
overlay  
br_netfilter  
EOF
```

6. Configure Kubernetes IPv4 networking (on all servers)

It's important to configure Kubernetes networking so that pods can communicate with each other and outside environments without a hitch.

Create a Kubernetes configuration file in the **/etc/sysctl.d/** directory and run commands:

```
# sudo nano /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
```

7. Save and exit. Then apply the settings by running the following command:

```
# sudo sysctl --system
```

8. Install dependencies & important packages

Command :

```
sudo apt install -y curl gnupg2 software-properties-common apt-transport-https ca-
certificates
```

9. Enable the Docker Repository

Command:

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/trusted.gpg.d/docker.gpg
sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu
$(lsb_release -cs) stable"
```

10. Install Containerd Runtime (on all servers)

containerd is a lightweight container runtime that manages the complete container lifecycle on a system. It is used by platforms like Docker and Kubernetes to manage containers efficiently.

```
# sudo apt update
```

```
# sudo apt install -y containerd.io
```

11. Configure containerd to start using systemd as cgroup

```
# Command
```

```
containerd config default | sudo tee /etc/containerd/config.toml >/dev/null 2>&1  
sudo sed -i 's/SystemdCgroup \= false/SystemdCgroup \= true/g' /etc/containerd/config.toml
```

12. Restart and enable the containerd service

```
# sudo systemctl restart containerd
```

```
# sudo systemctl enable containerd
```

13. Install & Setup Kubernetes Components (on all servers)

Add the Kubernetes GPG signing key.

Command

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.33/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

Add the official Kubernetes repository to your system.

Command

```
echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.33/deb/ /" | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

Get Updates

sudo apt update

To install kubeadm, kubectl, kubelet

sudo apt install kubelet kubeadm kubectl -y

Initialize Kubernetes Cluster (on master server)

The next step is to initialize the Kubernetes cluster on the master node. This configures the master node as the control plane. To initialize the cluster, run the command shown.

The **--pod-network-cidr** indicates a unique pod network for the cluster, in this case, the **10.10.0.0** network with a CIDR of **/16**.

```
# sudo kubeadm init --pod-network-cidr=10.10.0.0/16
```

To setup kubeconfig on master-server

Command

```
mkdir -p $HOME/.kube  
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config  
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

```
Your Kubernetes control-plane has initialized successfully!  
  
To start using your cluster, you need to run the following as a regular user  
:  
  
mkdir -p $HOME/.kube  
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config  
sudo chown $(id -u):$(id -g) $HOME/.kube/config  
  
Alternatively, if you are the root user, you can run:  
  
export KUBECONFIG=/etc/kubernetes/admin.conf  
  
You should now deploy a pod network to the cluster.  
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:  
https://kubernetes.io/docs/concepts/cluster-administration/addons/  
  
Then you can join any number of worker nodes by running the following on each as root:  
  
kubeadm join :6443 --token rg70py.y6bt1roknt8uwlos \  
--discovery-token-ca-cert-hash sha256:c1af33fc3c69d964875b5a0e85b42e09827262e92629e9adb037c3eccc47424
```

Add worker servers to the cluster (worker nodes)

Command

kubeadm join <master-ip>:6443 --token <your-token> --discovery-token-ca-cert-hash <your-hash>

```
Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user
:

mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on each
as root:

kubeadm join <master-ip>:6443 --token rg70py.y6bt1roknt8uwlos \
--discovery-token-ca-cert-hash sha256:c1af33fc3c69d964875b5a0e85b42e
09827262e92629e9adb037c3eccc47424
```

List nodes (on master-server)

Command

kubectl get nodes

NAME	STATUS	ROLES	AGE	VERSION
master	NotReady	control-plane	33m	v1.33.4
server1	NotReady	<none>	13s	v1.33.4

Install Kubernetes Network Plugin (on master-server)

To enable communication between pods in the cluster, you need a network plugin. You can install **Calico** or **Cilium** network plugin.

Calico Command

```
kubectl apply -f
```

```
https://raw.githubusercontent.com/projectcalico/calico/v3.30.0/manifests/calico.yaml
```

Cilium Command

```
helm upgrade cilium cilium/cilium --version 1.18.1 \
--namespace kube-system \
--reuse-values \
--set hubble.relay.enabled=true
```

Verification and testing the Cluster (on master-server)

Command

```
kubectl get pods -n kube-system
```

```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	control-plane	82m	v1.33.4
server1	Ready	<none>	49m	v1.33.4

Expose UI using Hubble UI

1. Enable Hubble UI

If installed using CLI

`cilium hubble enable`

If installed using Helm

```
helm upgrade cilium cilium/cilium --version 1.18.1 \
--namespace kube-system \
--reuse-values \
--set hubble.relay.enabled=true
```

2. Check the status

Command

`cilium status`

```
ubuntu@master:~$ cilium status
Cilium: OK
Operator: OK
Envoy DaemonSet: OK
Hubble Relay: OK
ClusterMesh: disabled

DaemonSet cilium Desired: 2, Ready: 2/2, Available: 2/2
DaemonSet cilium-envoy Desired: 2, Ready: 2/2, Available: 2/2
Deployment cilium-operator Desired: 1, Ready: 1/1, Available: 1/1
Deployment hubble-relay Desired: 1, Ready: 1/1, Available: 1/1
Deployment hubble-ui Desired: 1, Ready: 1/1, Available: 1/1
Containers cilium Running: 2
cilium-envoy Running: 2
cilium-operator Running: 1
clustermesh-apiserver Running: 1
hubble-relay Running: 1
hubble-ui Running: 1

Cluster Pods: 5/5 managed by Cilium
Helm chart version: 1.18.1
Image versions
526194dda08124f5eded833bf02bf: 2
cilium quay.io/cilium/cilium:v1.18.0@sha256:dfea023972d06ec183cfa3c9e7809716f85daaff042e573ef366e9ec6a0c0ab2: 2
cilium-envoy quay.io/cilium/cilium-envoy:v1.34.4-1753677767-266d5a01d1d55bd1d60148f991b98dac0390d363@sha256:231b5bd9682dfc648ae97f33dcdc5225c5a
cilium-operator-generic:v1.18.0@sha256:398378b4507b6e9db22be2f4455d8f6e509b189470061b0f613f0fabaf944f51: 1
cilium-operator quay.io/cilium/hubble-relay:v1.18.0@sha256:c13679f22ed250457b7f3581189d97f035608fe13c87b51f57f8a755918e793a: 1
cilium-relay quay.io/cilium/hubble-ui-backend:v0.13.2@sha256:a034b7e98e6ea796ed26df8f4e71f83fc16465a19d166eff67a03b822c0bfa15: 1
cilium-ui quay.io/cilium/hubble-ui:v0.13.2@sha256:9e37c1296b802830834cc87342a9182ccbb71ffeb711971e849221bd9d59392: 1
```

